

```

// server/index.ts
import express2 from "express";

// server/routes.ts
import { createServer } from "http";

// server/db.ts
import { createClient } from "@supabase/supabase-js";
var supabaseUrl = process.env.SUPABASE_URL;
var supabaseKey = process.env.SUPABASE_KEY;
if (!supabaseUrl || !supabaseKey) {
  throw new Error(
    "SUPABASE_URL e SUPABASE_KEY devem estar definidos. Configure as variáveis de ambiente corretamente."
  );
}
if (!supabaseUrl.match(/https:\/\/[^\.]+\supabase\.co/)) {
  console.warn(
    "Aviso: Formato de SUPABASE_URL não segue o padrão esperado https://[project-ref].supabase.co"
  );
}
var supabase = createClient(supabaseUrl, supabaseKey, {
  auth: {
    autoRefreshToken: false,
    persistSession: false
  },
  global: {
    headers: {
      "X-Client-Info": "supabase-js-client"
    }
  }
});

// server/database-storage.ts
import { randomUUID } from "crypto";
var DatabaseStorage = class {
  // User & Profile
  async getUser(id) {
    const { data, error } = await supabase.from("users").select("*").eq("id", id).single();
    if (error || !data) return void 0;
    return data;
  }
  async getUserByUsername(username) {
    const { data, error } = await supabase.from("users").select("*").eq("username", username).single();
    if (error || !data) return void 0;
    return data;
  }
  async createUser(user) {

```

```

    const { data, error } = await supabase.from("users").insert(user).select().single();
    if (error) throw error;
    return data;
}
async getProfile(id) {
    const { data, error } = await supabase.from("profiles").select("*").eq("id", id).single();
    if (error || !data) return void 0;
    return data;
}
async createProfile(profile) {
    const { data, error } = await supabase.from("profiles").insert(profile).select().single();
    if (error) throw error;
    return data;
}
// Clients
async getClients() {
    const { data, error } = await supabase.from("clients").select("*").order("name");
    if (error) throw error;
    return data;
}
async getClient(id) {
    const { data, error } = await supabase.from("clients").select("*").eq("id", id).single();
    if (error || !data) return void 0;
    return data;
}
async createClient(client) {
    const clientData = {
        ...client,
        id: client.id || randomUUID(),
        created_at: client.created_at || /* @__PURE__ */ new Date()
    };
    const { data, error } = await supabase.from("clients").insert(clientData).select().single();
    if (error) throw error;
    return data;
}
async updateClient(id, client) {
    const { data, error } = await supabase.from("clients").update(client).eq("id", id).select().single();
    if (error) throw error;
    return data;
}
// Services
async getServices() {
    const { data, error } = await supabase.from("services").select("*").order("name");
    if (error) throw error;
    return data;
}
async getService(id) {

```

```

    const { data, error } = await supabase.from("services").select("*").eq("id", id).single();
    if (error || !data) return void 0;
    return data;
}
async createService(service) {
    const serviceData = {
        ...service,
        id: service.id || randomUUID(),
        created_at: service.created_at || /* @__PURE__ */ new Date()
    };
    const { data, error } = await supabase.from("services").insert(serviceData).select().single();
    if (error) throw error;
    return data;
}
async updateService(id, service) {
    const { data, error } = await supabase.from("services").update(service).eq("id", id).select().single();
    if (error) throw error;
    return data;
}
async deleteService(id) {
    const { error } = await supabase.from("services").delete().eq("id", id);
    return !error;
}
// Appointments
async getAllAppointments(startDate, endDate) {
    let query = supabase.from("appointments").select("*", clients("*")).order("start_time", { ascending: false });
    if (startDate) {
        query = query.gte("start_time", startDate.toISOString());
    }
    if (endDate) {
        query = query.lte("start_time", endDate.toISOString());
    }
    const { data, error } = await query;
    if (error) throw error;
    const appointments2 = data?.map((appointment) => {
        const { clients: clients2, ...appointmentData } = appointment;
        return {
            ...appointmentData,
            client: clients2,
            // Mapeia clients para client (singular)
            client_name: clients2?.name || "",
            // Adiciona client_name explicitamente
            client_phone: clients2?.phone || ""
            // Adiciona client_phone explicitamente
        };
    });
    return appointments2;
}

```

```

}
// Mantemos o método original para compatibilidade
async getAppointments(startDate, endDate, page = 1, perPage = 20) {
  const offset = (page - 1) * perPage;
  let countQuery = supabase.from("appointments").select("id", { count: "exact" });
  if (startDate) {
    countQuery = countQuery.gte("start_time", startDate.toISOString());
  }
  if (endDate) {
    countQuery = countQuery.lte("start_time", endDate.toISOString());
  }
  const { count, error: countError } = await countQuery;
  if (countError) throw countError;
  let query = supabase.from("appointments").select("*, clients(*)").order("start_time", { ascending: false }).range(offset, offset + perPage -
1);
  if (startDate) {
    query = query.gte("start_time", startDate.toISOString());
  }
  if (endDate) {
    query = query.lte("start_time", endDate.toISOString());
  }
  const { data, error } = await query;
  if (error) throw error;
  const appointments2 = data?.map((appointment) => {
    const { clients: clients2, ...appointmentData } = appointment;
    return {
      ...appointmentData,
      client: clients2,
      // Mapeia clients para client (singular)
      client_name: clients2?.name || "",
      // Adiciona client_name explicitamente
      client_phone: clients2?.phone || ""
      // Adiciona client_phone explicitamente
    };
  }) || [];
  return {
    appointments: appointments2,
    total: count || 0
  };
}

async getAppointmentWithServices(id) {
  const { data: appointment, error: appointmentError } = await supabase.from("appointments").select("*, clients(*)").eq("id", id).single();
  if (appointmentError || !appointment) throw appointmentError;
  const { data: appointmentServices2, error: servicesError } = await supabase.from("appointment_services").select("*,
services(*)").eq("appointment_id", id);
  if (servicesError) throw servicesError;
  const { clients: clients2, ...appointmentData } = appointment;

```

```

return {
  ...appointmentData,
  client: clients2,
  client_name: clients2?.name || "",
  client_phone: clients2?.phone || "",
  services: appointmentServices2
};
}
}
async getUpcomingAppointments(limit = 10) {
  const today = /* __PURE__ */ new Date();
  const { data, error } = await supabase.from("appointments").select("*", clients(*)).gte("start_time", today.toISOString()).eq("status",
"scheduled").order("start_time").limit(limit);
  if (error) throw error;
  const result = [];
  for (const appointment of data) {
    const { data: services2, error: servicesError } = await supabase.from("appointment_services").select("*",
services(*)).eq("appointment_id", appointment.id);
    if (servicesError) throw servicesError;
    const { clients: clients2, ...appointmentData } = appointment;
    result.push({
      ...appointmentData,
      client: clients2,
      // Mapeia clients para client (singular)
      client_name: clients2?.name || "",
      // Adiciona client_name explicitamente
      client_phone: clients2?.phone || "",
      // Adiciona client_phone explicitamente
      services: services2
    });
  }
  return result;
}
}
async createAppointment(appointment, serviceIds) {
  const appointmentData = {
    ...appointment,
    id: appointment.id || randomUUID(),
    created_at: appointment.created_at || /* __PURE__ */ new Date(),
    payment_status: appointment.payment_status || "pending"
  };
  const { data, error } = await supabase.from("appointments").insert(appointmentData).select().single();
  if (error) throw error;
  if (serviceIds.length > 0) {
    const { data: servicesData, error: servicesError } = await supabase.from("services").select("*").in("id", serviceIds);
    if (servicesError) throw servicesError;
    const appointmentServicesData = servicesData.map((service) => ({
      id: randomUUID(),
      appointment_id: data.id,

```

```

        service_id: service.id,
        price: service.price,
        final_price: service.price
    }));
    const { error: insertError } = await supabase.from("appointment_services").insert(appointmentServicesData);
    if (insertError) throw insertError;
    const totalPrice = appointmentServicesData.reduce(
        (sum, item) => sum + parseFloat(item.price),
        0
    ).toString();
    await supabase.from("appointments").update({ final_price: totalPrice }).eq("id", data.id);
}
return data;
}
async updateAppointment(id, appointment) {
    const { data, error } = await supabase.from("appointments").update(appointment).eq("id", id).select().single();
    if (error) throw error;
    return data;
}
async deleteAppointment(id) {
    const { error: servicesError } = await supabase.from("appointment_services").delete().eq("appointment_id", id);
    if (servicesError) throw servicesError;
    const { error } = await supabase.from("appointments").delete().eq("id", id);
    return !error;
}
// Appointment Services
async getAppointmentServices(appointmentId) {
    const { data, error } = await supabase.from("appointment_services").select("*, services(*)").eq("appointment_id", appointmentId);
    if (error) throw error;
    return data;
}
async createAppointmentService(service) {
    const serviceData = {
        ...service,
        id: service.id || randomUUID(),
        final_price: service.final_price || service.price
    };
    const { data, error } = await supabase.from("appointment_services").insert(serviceData).select().single();
    if (error) throw error;
    return data;
}
// Blocked Schedules
async getBlockedSchedules(startDate, endDate) {
    let query = supabase.from("blocked_schedules").select("*").order("start_time");
    if (startDate) {
        query = query.gte("start_time", startDate.toISOString());
    }
}

```

```

    if (endDate) {
      query = query.lte("start_time", endDate.toISOString());
    }
    const { data, error } = await query;
    if (error) throw error;
    return data;
  }
  async createBlockedSchedule(blockedSchedule) {
    const scheduleData = {
      ...blockedSchedule,
      id: blockedSchedule.id || randomUUID(),
      created_at: blockedSchedule.created_at || /* __PURE__ */ new Date(),
      reason: blockedSchedule.reason || "Hor\xE1rio bloqueado"
    };
    const { data, error } = await supabase.from("blocked_schedules").insert(scheduleData).select().single();
    if (error) throw error;
    return data;
  }
  async deleteBlockedSchedule(id) {
    const { error } = await supabase.from("blocked_schedules").delete().eq("id", id);
    return !error;
  }
  // Inventory
  async getInventory() {
    const { data, error } = await supabase.from("inventory").select("*").order("name");
    if (error) throw error;
    return data;
  }
  async getLowStockItems(limit = 10) {
    const { data, error } = await supabase.from("inventory").select("*").lt("quantity", 5).order("quantity").limit(limit);
    if (error) throw error;
    return data;
  }
  async getInventoryItem(id) {
    const { data, error } = await supabase.from("inventory").select("*").eq("id", id).single();
    if (error || !data) return void 0;
    return data;
  }
  async createInventoryItem(item) {
    const itemData = {
      ...item,
      id: item.id || randomUUID(),
      created_at: item.created_at || /* __PURE__ */ new Date(),
      category: item.category || "Geral"
    };
    const { data, error } = await supabase.from("inventory").insert(itemData).select().single();
    if (error) throw error;
  }

```

```

    return data;
}
async updateInventoryItem(id, item) {
    const { data, error } = await supabase.from("inventory").update(item).eq("id", id).select().single();
    if (error) throw error;
    return data;
}
async updateInventoryQuantity(id, quantity) {
    const { data: currentItem, error: getError } = await supabase.from("inventory").select("quantity").eq("id", id).single();
    if (getError) throw getError;
    const newQuantity = (currentItem.quantity || 0) + quantity;
    const { data, error } = await supabase.from("inventory").update({ quantity: newQuantity }).eq("id", id).select().single();
    if (error) throw error;
    return data;
}
// Sales
async getSales(startDate, endDate) {
    let query = supabase.from("sales").select("*, clients(*)").order("sale_date", { ascending: false });
    if (startDate) {
        query = query.gte("sale_date", startDate.toISOString());
    }
    if (endDate) {
        query = query.lte("sale_date", endDate.toISOString());
    }
    const { data, error } = await query;
    if (error) throw error;
    return data;
}
async getSale(id) {
    const { data, error } = await supabase.from("sales").select("*, clients(*)").eq("id", id).single();
    if (error || !data) return void 0;
    return data;
}
async getSaleWithItems(id) {
    const { data: sale, error: saleError } = await supabase.from("sales").select("*, clients(*)").eq("id", id).single();
    if (saleError || !sale) throw saleError;
    const { data: saleItems2, error: itemsError } = await supabase.from("sale_items").select("*, inventory(*)").eq("sale_id", id);
    if (itemsError) throw itemsError;
    return {
        ...sale,
        items: saleItems2
    };
}
async createSale(sale, items) {
    const saleData = {
        ...sale,
        id: sale.id || randomUUID(),
    }

```



```

    created_at: sale.created_at || /* @__PURE__ */ new Date(),
    sale_date: sale.sale_date || /* @__PURE__ */ new Date()
  };
  if (!saleData.total_amount) {
    saleData.total_amount = items.reduce((sum, item) => sum + item.quantity * item.unitPrice, 0).toString();
  }
  const { data, error } = await supabase.from("sales").insert(saleData).select().single();
  if (error) throw error;
  if (items.length > 0) {
    const saleItemsData = items.map((item) => ({
      id: randomUUID(),
      sale_id: data.id,
      inventory_id: item.inventoryId,
      quantity: item.quantity,
      unit_price: item.unitPrice.toString()
    }));
    const { error: itemsError } = await supabase.from("sale_items").insert(saleItemsData);
    if (itemsError) throw itemsError;
    for (const item of items) {
      await this.updateInventoryQuantity(item.inventoryId, -item.quantity);
    }
    await this.createFinancialTransaction({
      transaction_date: /* @__PURE__ */ new Date(),
      description: `Venda de produtos - ${data.id}`,
      amount: saleData.total_amount,
      type: "income",
      category: "Vendas",
      payment_method: saleData.payment_method || "Dinheiro",
      related_sale_id: data.id,
      created_at: /* @__PURE__ */ new Date()
    });
  }
  return data;
}

// Financial Transactions
async getFinancialTransactions(startDate, endDate, type) {
  let query = supabase.from("financial_transactions").select("*").order("transaction_date", { ascending: false });
  if (startDate) {
    query = query.gte("transaction_date", startDate.toISOString());
  }
  if (endDate) {
    query = query.lte("transaction_date", endDate.toISOString());
  }
  if (type) {
    query = query.eq("type", type);
  }
  const { data, error } = await query;

```

```

    if (error) throw error;
    return data;
}
async createFinancialTransaction(transaction) {
  const transactionData = {
    ...transaction,
    id: transaction.id || randomUUID(),
    created_at: transaction.created_at || /* #__PURE__ */ new Date(),
    transaction_date: transaction.transaction_date || /* #__PURE__ */ new Date()
  };
  const { data, error } = await supabase.from("financial_transactions").insert(transactionData).select().single();
  if (error) throw error;
  return data;
}
async getFinancialSummary(startDate, endDate) {
  const { data: incomes, error: incomesError } = await supabase.from("financial_transactions").select("amount").eq("type", "income").gte("transaction_date", startDate.toISOString()).lte("transaction_date", endDate.toISOString());
  if (incomesError) throw incomesError;
  const { data: expenses, error: expensesError } = await supabase.from("financial_transactions").select("amount").eq("type", "expense").gte("transaction_date", startDate.toISOString()).lte("transaction_date", endDate.toISOString());
  if (expensesError) throw expensesError;
  const totalIncome = incomes.reduce(
    (sum, item) => sum + parseFloat(item.amount),
    0
  );
  const totalExpense = expenses.reduce(
    (sum, item) => sum + parseFloat(item.amount),
    0
  );
  const { data: categorySummary, error: categoryError } = await supabase.from("financial_transactions").select("category, type, amount").gte("transaction_date", startDate.toISOString()).lte("transaction_date", endDate.toISOString());
  if (categoryError) throw categoryError;
  const categoriesMap = categorySummary.reduce((acc, item) => {
    const category = item.category || "Outros";
    if (!acc[category]) {
      acc[category] = { income: 0, expense: 0 };
    }
    if (item.type === "income") {
      acc[category].income += parseFloat(item.amount);
    } else {
      acc[category].expense += parseFloat(item.amount);
    }
  }, {});
  return acc;
}, {});
const categories = Object.keys(categoriesMap).map((category) => ({
  category,
  income: categoriesMap[category].income,

```

```

        expense: categoriesMap[category].expense,
        balance: categoriesMap[category].income - categoriesMap[category].expense
    }));
    return {
        totalIncome,
        totalExpense,
        balance: totalIncome - totalExpense,
        categories
    };
}
};

// server/storage.ts
var storage = new DatabaseStorage();

// server/routes.ts
import { z as z2 } from "zod";

// server/supabase-admin.ts
import { createClient as createClient2 } from "@supabase/supabase-js";
var supabaseUrl2 = process.env.SUPABASE_URL;
var supabaseKey2 = process.env.SUPABASE_KEY;
console.log(`Supabase URL dispon\xE1vel: ${!!supabaseUrl2}`);
console.log(`Supabase Key dispon\xE1vel: ${!!supabaseKey2}`);
if (supabaseUrl2) console.log(`Supabase URL come\xE7a com: ${supabaseUrl2.substring(0, 8)}...`);
if (supabaseKey2) console.log(`Supabase Key come\xE7a com: ${supabaseKey2.substring(0, 5)}...`);
if (!supabaseUrl2 || !supabaseKey2) {
    console.error("As vari\xE1veis de ambiente SUPABASE_URL e SUPABASE_KEY s\xE3o necess\xE1rias para a conex\xE3o com o Supabase.");
}
var supabaseAdmin = createClient2(supabaseUrl2, supabaseKey2, {
    auth: {
        autoRefreshToken: false,
        persistSession: false
    },
    // Criando cabeçalhos personalizados para garantir que estamos usando a chave de serviço
    global: {
        headers: {
            // Adicionando o header que identifica esta chave como uma chave de serviço (service role)
            "X-Client-Info": "supabase-js-admin"
        }
    }
});
});
async function syncDataWithSupabase() {
    try {
        console.log("Iniciando sincroniza\xE7\xE3o com Supabase...");
        const results = {
            profiles: { success: false, count: 0, message: "" },

```

```

users: { success: false, count: 0, message: "" },
clients: { success: false, count: 0, message: "" },
appointments: { success: false, count: 0, message: "" },
operations: []
};
try {
  const { data: profiles2, error: profilesError } = await supabaseAdmin.from("profiles").select("*");
  if (profilesError) {
    results.profiles.message = profilesError.message;
    console.error("Erro ao buscar perfis do Supabase:", profilesError);
  } else if (profiles2) {
    results.profiles.success = true;
    results.profiles.count = profiles2.length;
    results.operations.push(`Encontrados ${profiles2.length} perfis no Supabase`);
    console.log(`Encontrados ${profiles2.length} perfis no Supabase`);
  }
} catch (profileError) {
  results.profiles.message = profileError?.message || "Erro desconhecido ao sincronizar perfis";
  console.error("Erro ao processar perfis:", profileError);
}
try {
  const { data: usersData, error: usersError } = await supabaseAdmin.auth.admin.listUsers();
  if (usersError) {
    results.users.message = usersError.message;
    console.error("Erro ao buscar usu\u00e1rios do Supabase:", usersError);
  } else if (usersData) {
    results.users.success = true;
    results.users.count = usersData.users.length;
    results.operations.push(`Encontrados ${usersData.users.length} usu\u00e1rios no Supabase`);
    console.log(`Encontrados ${usersData.users.length} usu\u00e1rios no Supabase`);
  }
} catch (usersError) {
  results.users.message = usersError?.message || "Erro desconhecido ao sincronizar usu\u00e1rios";
  console.error("Erro ao processar usu\u00e1rios:", usersError);
}
try {
  const { data: clients2, error: clientsError } = await supabaseAdmin.from("clients").select("*");
  if (clientsError) {
    results.clients.message = clientsError.message;
    console.error("Erro ao buscar clientes do Supabase:", clientsError);
  } else if (clients2) {
    results.clients.success = true;
    results.clients.count = clients2.length;
    results.operations.push(`Encontrados ${clients2.length} clientes no Supabase`);
    console.log(`Encontrados ${clients2.length} clientes no Supabase`);
  }
} catch (clientsError) {

```

```

    results.clients.message = clientsError?.message || "Erro desconhecido ao sincronizar clientes";
    console.error("Erro ao processar clientes:", clientsError);
  }
  try {
    const { data: appointments2, error: appointmentsError } = await supabaseAdmin.from("appointments").select("*");
    if (appointmentsError) {
      results.appointments.message = appointmentsError.message;
      console.error("Erro ao buscar agendamentos do Supabase:", appointmentsError);
    } else if (appointments2) {
      results.appointments.success = true;
      results.appointments.count = appointments2.length;
      results.operations.push(`Encontrados ${appointments2.length} agendamentos no Supabase`);
      console.log(`Encontrados ${appointments2.length} agendamentos no Supabase`);
    }
  } catch (appointmentsError) {
    results.appointments.message = appointmentsError?.message || "Erro desconhecido ao sincronizar agendamentos";
    console.error("Erro ao processar agendamentos:", appointmentsError);
  }
  const anySuccess = results.profiles.success || results.users.success || results.clients.success || results.appointments.success;
  console.log(`Sincroniza com Supabase ${anySuccess ? "concluiu" : "falhou"}`);
  return {
    success: anySuccess,
    results,
    timestamp: (/* @__PURE__ */ new Date()).toISOString()
  };
} catch (error) {
  console.error("Erro geral ao sincronizar com Supabase:", error);
  return {
    success: false,
    error: error?.message || "Erro desconhecido",
    timestamp: (/* @__PURE__ */ new Date()).toISOString()
  };
}
}

async function getSupabaseClients() {
  try {
    const { data, error } = await supabaseAdmin.from("clients").select("*");
    if (error) {
      console.error("Erro ao buscar clientes do Supabase:", error);
      return { success: false, error: error.message };
    }
    return {
      success: true,
      clients: data || [],
      count: data?.length || 0
    };
  } catch (error) {

```

```

    console.error("Erro ao buscar clientes do Supabase:", error);
    return {
      success: false,
      error: error?.message || "Erro desconhecido"
    };
  }
}
}
async function getSupabaseAppointments() {
  try {
    const { data, error } = await supabaseAdmin.from("appointments").select("*");
    if (error) {
      console.error("Erro ao buscar agendamentos do Supabase:", error);
      return { success: false, error: error.message };
    }
    return {
      success: true,
      appointments: data || [],
      count: data?.length || 0
    };
  } catch (error) {
    console.error("Erro ao buscar agendamentos do Supabase:", error);
    return {
      success: false,
      error: error?.message || "Erro desconhecido"
    };
  }
}
}

// shared/schema.ts
import { pgTable, text, serial, integer, numeric, timestamp, uuid } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod";
var clients = pgTable("clients", {
  id: uuid("id").defaultRandom().primaryKey(),
  name: text("name").notNull(),
  phone: text("phone"),
  created_at: timestamp("created_at").defaultNow(),
  created_by: uuid("created_by")
});
var insertClientSchema = createInsertSchema(clients).pick({
  name: true,
  phone: true
});
var services = pgTable("services", {
  id: uuid("id").defaultRandom().primaryKey(),
  name: text("name").notNull(),
  price: numeric("price").notNull().default("0"),

```

```

    duration: integer("duration").notNull().default(60),
    created_at: timestamp("created_at").defaultNow(),
    created_by: uuid("created_by")
  });
var insertServiceSchema = createInsertSchema(services).pick({
  name: true,
  price: true,
  duration: true
});
var appointments = pgTable("appointments", {
  id: uuid("id").defaultRandom().primaryKey(),
  client_id: uuid("client_id").notNull().references(() => clients.id),
  start_time: timestamp("start_time").notNull(),
  end_time: timestamp("end_time").notNull(),
  status: text("status").notNull().default("agendado"),
  notes: text("notes"),
  created_at: timestamp("created_at").defaultNow(),
  created_by: uuid("created_by"),
  final_price: numeric("final_price").default("0"),
  recurrence: text("recurrence"),
  payment_date: timestamp("payment_date"),
  payment_status: text("payment_status")
});
var insertAppointmentSchema = createInsertSchema(appointments, {
  start_time: z.coerce.date(),
  // Aceita string e converte para Date
  end_time: z.coerce.date(),
  // Aceita string e converte para Date
  payment_status: z.string().nullable()
  // Permitir null no payment_status
}).pick({
  client_id: true,
  start_time: true,
  end_time: true,
  status: true,
  notes: true,
  final_price: true,
  recurrence: true,
  payment_status: true
});
var appointmentServices = pgTable("appointment_services", {
  id: uuid("id").defaultRandom().primaryKey(),
  appointment_id: uuid("appointment_id").notNull().references(() => appointments.id, { onDelete: "cascade" }),
  service_id: uuid("service_id").notNull().references(() => services.id),
  price: numeric("price").notNull(),
  final_price: numeric("final_price").default("0")
});

```

```

var insertAppointmentServiceSchema = createInsertSchema(appointmentServices).pick({
  appointment_id: true,
  service_id: true,
  price: true,
  final_price: true
});
var blockedSchedules = pgTable("blocked_schedules", {
  id: uuid("id").defaultRandom().primaryKey(),
  start_time: timestamp("start_time").notNull(),
  end_time: timestamp("end_time").notNull(),
  reason: text("reason"),
  created_at: timestamp("created_at").defaultNow()
});
var insertBlockedScheduleSchema = createInsertSchema(blockedSchedules, {
  start_time: z.coerce.date(),
  // Aceita string e converte para Date
  end_time: z.coerce.date(),
  // Aceita string e converte para Date
}).pick({
  start_time: true,
  end_time: true,
  reason: true
});
var inventory = pgTable("inventory", {
  id: uuid("id").defaultRandom().primaryKey(),
  name: text("name").notNull(),
  quantity: integer("quantity").notNull().default(0),
  cost_price: numeric("cost_price").notNull().default("0"),
  selling_price: numeric("selling_price").notNull().default("0"),
  created_at: timestamp("created_at").defaultNow(),
  created_by: uuid("created_by"),
  category: text("category").default("Geral")
});
var insertInventorySchema = createInsertSchema(inventory).pick({
  name: true,
  quantity: true,
  cost_price: true,
  selling_price: true,
  category: true
});
var sales = pgTable("sales", {
  id: uuid("id").defaultRandom().primaryKey(),
  sale_date: timestamp("sale_date").defaultNow().notNull(),
  total_amount: numeric("total_amount").default("0").notNull(),
  payment_method: text("payment_method"),
  notes: text("notes"),
  created_at: timestamp("created_at").defaultNow(),

```



```

    created_by: uuid("created_by"),
    client_id: uuid("client_id").references(() => clients.id)
  });
var insertSaleSchema = createInsertSchema(sales, {
  sale_date: z.coerce.date()
  // Aceita string e converte para Date
}).pick({
  sale_date: true,
  total_amount: true,
  payment_method: true,
  notes: true,
  client_id: true
});
var saleItems = pgTable("sale_items", {
  id: uuid("id").defaultRandom().primaryKey(),
  sale_id: uuid("sale_id").notNull().references(() => sales.id, { onDelete: "cascade" }),
  inventory_id: uuid("inventory_id").notNull().references(() => inventory.id),
  quantity: integer("quantity").default(1).notNull(),
  unit_price: numeric("unit_price").default("0").notNull(),
  total_price: numeric("total_price").default("0").notNull()
});
var insertSaleItemSchema = createInsertSchema(saleItems).pick({
  sale_id: true,
  inventory_id: true,
  quantity: true,
  unit_price: true,
  total_price: true
});
var financialTransactions = pgTable("financial_transactions", {
  id: uuid("id").defaultRandom().primaryKey(),
  transaction_date: timestamp("transaction_date").defaultNow().notNull(),
  description: text("description").notNull(),
  amount: numeric("amount").notNull(),
  type: text("type").notNull(),
  // income or expense
  category: text("category"),
  related_sale_id: uuid("related_sale_id").references(() => sales.id),
  related_appointment_id: uuid("related_appointment_id").references(() => appointments.id),
  payment_method: text("payment_method"),
  notes: text("notes"),
  created_at: timestamp("created_at").defaultNow(),
  created_by: uuid("created_by")
});
var insertFinancialTransactionSchema = createInsertSchema(financialTransactions, {
  transaction_date: z.coerce.date()
  // Aceita string e converte para Date
}).pick({

```

```

    transaction_date: true,
    description: true,
    amount: true,
    type: true,
    category: true,
    related_sale_id: true,
    related_appointment_id: true,
    payment_method: true,
    notes: true
  });
var users = pgTable("users", {
  id: serial("id").primaryKey(),
  username: text("username").notNull().unique(),
  password: text("password").notNull()
});
var insertUserSchema = createInsertSchema(users).pick({
  username: true,
  password: true
});
var profiles = pgTable("profiles", {
  id: uuid("id").primaryKey(),
  email: text("email").notNull().unique(),
  name: text("name"),
  created_at: timestamp("created_at").defaultNow()
});
var insertProfileSchema = createInsertSchema(profiles).pick({
  id: true,
  email: true,
  name: true
});

// server/routes.ts
async function registerRoutes(app2) {
  app2.get("/api/clients", async (req, res) => {
    try {
      const clients2 = await storage.getClients();
      res.json(clients2);
    } catch (error) {
      res.status(500).json({ message: "Erro ao buscar clientes" });
    }
  });
  app2.get("/api/clients/:id", async (req, res) => {
    try {
      const client = await storage.getClient(req.params.id);
      if (!client) {
        return res.status(404).json({ message: "Cliente não encontrado" });
      }
    }
  });
}

```

```

    res.json(client);
  } catch (error) {
    res.status(500).json({ message: "Erro ao buscar cliente" });
  }
});
app2.post("/api/clients", async (req, res) => {
  try {
    const validatedData = insertClientSchema.parse(req.body);
    const client = await storage.createClient(validatedData);
    res.status(201).json(client);
  } catch (error) {
    if (error instanceof z2.ZodError) {
      return res.status(400).json({ message: "Dados inv\u00e1lidos", errors: error.errors });
    }
    res.status(500).json({ message: "Erro ao criar cliente" });
  }
});
app2.put("/api/clients/:id", async (req, res) => {
  try {
    const validatedData = insertClientSchema.partial().parse(req.body);
    const client = await storage.updateClient(req.params.id, validatedData);
    if (!client) {
      return res.status(404).json({ message: "Cliente n\u00e3o encontrado" });
    }
    res.json(client);
  } catch (error) {
    if (error instanceof z2.ZodError) {
      return res.status(400).json({ message: "Dados inv\u00e1lidos", errors: error.errors });
    }
    res.status(500).json({ message: "Erro ao atualizar cliente" });
  }
});
app2.get("/api/services", async (req, res) => {
  try {
    const services2 = await storage.getServices();
    res.json(services2);
  } catch (error) {
    res.status(500).json({ message: "Erro ao buscar servi\u00e7os" });
  }
});
app2.get("/api/services/:id", async (req, res) => {
  try {
    const service = await storage.getService(req.params.id);
    if (!service) {
      return res.status(404).json({ message: "Servi\u00e7o n\u00e3o encontrado" });
    }
    res.json(service);
  }
});

```

```

    } catch (error) {
      res.status(500).json({ message: "Erro ao buscar servi\xE7o" });
    }
  });
app2.post("/api/services", async (req, res) => {
  try {
    const validatedData = insertServiceSchema.parse(req.body);
    const service = await storage.createService(validatedData);
    res.status(201).json(service);
  } catch (error) {
    if (error instanceof z2.ZodError) {
      return res.status(400).json({ message: "Dados inv\xE1lidos", errors: error.errors });
    }
    res.status(500).json({ message: "Erro ao criar servi\xE7o" });
  }
});
app2.put("/api/services/:id", async (req, res) => {
  try {
    const validatedData = insertServiceSchema.partial().parse(req.body);
    const service = await storage.updateService(req.params.id, validatedData);
    if (!service) {
      return res.status(404).json({ message: "Servi\xE7o n\xE3o encontrado" });
    }
    res.json(service);
  } catch (error) {
    if (error instanceof z2.ZodError) {
      return res.status(400).json({ message: "Dados inv\xE1lidos", errors: error.errors });
    }
    res.status(500).json({ message: "Erro ao atualizar servi\xE7o" });
  }
});
app2.delete("/api/services/:id", async (req, res) => {
  try {
    const deleted = await storage.deleteService(req.params.id);
    if (!deleted) {
      return res.status(404).json({ message: "Servi\xE7o n\xE3o encontrado" });
    }
    res.status(204).send();
  } catch (error) {
    res.status(500).json({ message: "Erro ao excluir servi\xE7o" });
  }
});
app2.get("/api/appointments", async (req, res) => {
  try {
    const startDate = req.query.startDate ? new Date(req.query.startDate) : void 0;
    const endDate = req.query.endDate ? new Date(req.query.endDate) : void 0;
    const allAppointments = await storage.getAllAppointments(startDate, endDate);

```

```

const enrichedAppointments = await Promise.all(
  allAppointments.map(async (appointment) => {
    return await storage.getAppointmentWithServices(appointment.id);
  })
);
res.json({
  appointments: enrichedAppointments,
  pagination: {
    total: enrichedAppointments.length,
    totalPages: Math.ceil(enrichedAppointments.length / 20)
    // Mantemos perPage=20 como referência
  }
});
} catch (error) {
  console.error("Erro detalhado ao buscar agendamentos:", error);
  res.status(500).json({
    message: "Erro ao buscar agendamentos",
    error: error instanceof Error ? error.message : String(error)
  });
}
});
app2.get("/api/appointments/upcoming", async (req, res) => {
  try {
    const limit = req.query.limit ? parseInt(req.query.limit) : void 0;
    const appointments2 = await storage.getUpcomingAppointments(limit);
    res.json(appointments2);
  } catch (error) {
    console.error("Erro detalhado ao buscar agendamentos pr\xF3ximos:", error);
    res.status(500).json({
      message: "Erro ao buscar agendamentos pr\xF3ximos",
      error: error instanceof Error ? error.message : String(error)
    });
  }
});
app2.get("/api/appointments/:id", async (req, res) => {
  try {
    console.log(`Buscando agendamento de ID: ${req.params.id}`);
    const appointment = await storage.getAppointmentWithServices(req.params.id);
    if (!appointment) {
      console.log(`Agendamento com ID ${req.params.id} n\xE3o encontrado`);
      return res.status(404).json({ message: "Agendamento n\xE3o encontrado" });
    }
    console.log(`Agendamento encontrado:`, JSON.stringify({
      id: appointment.id,
      client: appointment.client,
      services: appointment.services?.length || 0
    }));
  }
});

```

```

    res.json(appointment);
  } catch (error) {
    console.error("Erro ao buscar agendamento:", error);
    res.status(500).json({
      message: "Erro ao buscar agendamento",
      error: error instanceof Error ? error.message : String(error)
    });
  }
});
app2.post("/api/appointments", async (req, res) => {
  try {
    const { service_ids, ...appointmentData } = req.body;
    if (typeof appointmentData.start_time === "string") {
      appointmentData.start_time = new Date(appointmentData.start_time);
    }
    if (typeof appointmentData.end_time === "string") {
      appointmentData.end_time = new Date(appointmentData.end_time);
    }
    if (appointmentData.status === "scheduled") {
      appointmentData.status = "agendado";
    }
    if (!appointmentData.payment_status || appointmentData.payment_status === "pending") {
      appointmentData.payment_status = null;
    }
    const validatedAppointment = insertAppointmentSchema.parse(appointmentData);
    if (!Array.isArray(service_ids) || service_ids.length === 0) {
      return res.status(400).json({
        message: "Dados inv\u00e1lidos",
        errors: [{ path: ["service_ids"], message: "Selecione pelo menos um servi\u00e7o" }]
      });
    }
    console.log("Criando agendamento:", validatedAppointment, service_ids);
    const appointment = await storage.createAppointment(validatedAppointment, service_ids);
    res.status(201).json(appointment);
  } catch (error) {
    console.error("Erro ao criar agendamento:", error);
    if (error instanceof z.ZodError) {
      return res.status(400).json({ message: "Dados inv\u00e1lidos", errors: error.errors });
    }
    res.status(500).json({
      message: "Erro ao criar agendamento",
      error: error instanceof Error ? error.message : String(error)
    });
  }
});
app2.put("/api/appointments/:id", async (req, res) => {
  try {

```

```

const appointmentData = req.body;
if (typeof appointmentData.start_time === "string") {
  appointmentData.start_time = new Date(appointmentData.start_time);
}
if (typeof appointmentData.end_time === "string") {
  appointmentData.end_time = new Date(appointmentData.end_time);
}
if (appointmentData.status === "scheduled") {
  appointmentData.status = "agendado";
} else if (appointmentData.status === "cancelled" || appointmentData.status === "canceled") {
  appointmentData.status = "cancelado";
  appointmentData.payment_status = null;
} else if (appointmentData.status === "completed") {
  appointmentData.status = "finalizado";
} else if (appointmentData.status === "pending_payment") {
  appointmentData.status = "pagamento pendente";
}
if (appointmentData.payment_status === "paid") {
  appointmentData.payment_status = "pago";
  appointmentData.status = "finalizado";
} else if (appointmentData.payment_status === "pending") {
  appointmentData.payment_status = "pendente";
  appointmentData.status = "pagamento pendente";
}
const validatedData = insertAppointmentSchema.partial().parse(appointmentData);
const appointment = await storage.updateAppointment(req.params.id, validatedData);
if (!appointment) {
  return res.status(404).json({ message: "Agendamento n\u00e3o encontrado" });
}
res.json(appointment);
} catch (error) {
  console.error("Erro ao atualizar agendamento:", error);
  if (error instanceof z.ZodError) {
    return res.status(400).json({ message: "Dados inv\u00e1lidos", errors: error.errors });
  }
  res.status(500).json({
    message: "Erro ao atualizar agendamento",
    error: error instanceof Error ? error.message : String(error)
  });
}
});
app2.delete("/api/appointments/:id", async (req, res) => {
  try {
    const deleted = await storage.deleteAppointment(req.params.id);
    if (!deleted) {
      return res.status(404).json({ message: "Agendamento n\u00e3o encontrado" });
    }
  }

```

```

    res.status(204).send();
  } catch (error) {
    res.status(500).json({ message: "Erro ao excluir agendamento" });
  }
});
app2.get("/api/blocked-schedules", async (req, res) => {
  try {
    const startDate = req.query.startDate ? new Date(req.query.startDate) : void 0;
    const endDate = req.query.endDate ? new Date(req.query.endDate) : void 0;
    const blockedSchedules2 = await storage.getBlockedSchedules(startDate, endDate);
    res.json(blockedSchedules2);
  } catch (error) {
    res.status(500).json({ message: "Erro ao buscar hor\u00e1rios bloqueados" });
  }
});
app2.post("/api/blocked-schedules", async (req, res) => {
  try {
    const scheduleData = req.body;
    if (typeof scheduleData.start_time === "string") {
      scheduleData.start_time = new Date(scheduleData.start_time);
    }
    if (typeof scheduleData.end_time === "string") {
      scheduleData.end_time = new Date(scheduleData.end_time);
    }
    const validatedData = insertBlockedScheduleSchema.parse(scheduleData);
    const blockedSchedule = await storage.createBlockedSchedule(validatedData);
    res.status(201).json(blockedSchedule);
  } catch (error) {
    console.error("Erro ao bloquear hor\u00e1rio:", error);
    if (error instanceof z.ZodError) {
      return res.status(400).json({ message: "Dados inv\u00e1lidos", errors: error.errors });
    }
    res.status(500).json({
      message: "Erro ao bloquear hor\u00e1rio",
      error: error instanceof Error ? error.message : String(error)
    });
  }
});
app2.delete("/api/blocked-schedules/:id", async (req, res) => {
  try {
    const deleted = await storage.deleteBlockedSchedule(req.params.id);
    if (!deleted) {
      return res.status(404).json({ message: "Hor\u00e1rio bloqueado n\u00e3o encontrado" });
    }
    res.status(204).send();
  } catch (error) {
    res.status(500).json({ message: "Erro ao remover hor\u00e1rio bloqueado" });
  }
});

```



```

    }
  });
  app2.get("/api/inventory", async (req, res) => {
    try {
      const inventory2 = await storage.getInventory();
      res.json(inventory2);
    } catch (error) {
      res.status(500).json({ message: "Erro ao buscar estoque" });
    }
  });
  app2.get("/api/inventory/low-stock", async (req, res) => {
    try {
      const limit = req.query.limit ? parseInt(req.query.limit) : void 0;
      const lowStockItems = await storage.getLowStockItems(limit);
      res.json(lowStockItems);
    } catch (error) {
      res.status(500).json({ message: "Erro ao buscar itens com estoque baixo" });
    }
  });
  app2.get("/api/inventory/:id", async (req, res) => {
    try {
      const item = await storage.getInventoryItem(req.params.id);
      if (!item) {
        return res.status(404).json({ message: "Item não encontrado" });
      }
      res.json(item);
    } catch (error) {
      res.status(500).json({ message: "Erro ao buscar item do estoque" });
    }
  });
  app2.post("/api/inventory", async (req, res) => {
    try {
      const validatedData = insertInventorySchema.parse(req.body);
      const item = await storage.createInventoryItem(validatedData);
      res.status(201).json(item);
    } catch (error) {
      if (error instanceof z.ZodError) {
        return res.status(400).json({ message: "Dados inválidos", errors: error.errors });
      }
      res.status(500).json({ message: "Erro ao criar item no estoque" });
    }
  });
  app2.put("/api/inventory/:id", async (req, res) => {
    try {
      const validatedData = insertInventorySchema.partial().parse(req.body);
      const item = await storage.updateInventoryItem(req.params.id, validatedData);
      if (!item) {

```

```

    return res.status(404).json({ message: "Item n\u00E3o encontrado" });
  }
  res.json(item);
} catch (error) {
  if (error instanceof z2.ZodError) {
    return res.status(400).json({ message: "Dados inv\u00E1lidos", errors: error.errors });
  }
  res.status(500).json({ message: "Erro ao atualizar item do estoque" });
}
});
app2.put("/api/inventory/:id/quantity", async (req, res) => {
  try {
    const schema = z2.object({ quantity: z2.number().min(0) });
    const { quantity } = schema.parse(req.body);
    const item = await storage.updateInventoryQuantity(req.params.id, quantity);
    if (!item) {
      return res.status(404).json({ message: "Item n\u00E3o encontrado" });
    }
    res.json(item);
  } catch (error) {
    if (error instanceof z2.ZodError) {
      return res.status(400).json({ message: "Dados inv\u00E1lidos", errors: error.errors });
    }
    res.status(500).json({ message: "Erro ao atualizar quantidade do item" });
  }
});
app2.get("/api/sales", async (req, res) => {
  try {
    const startDate = req.query.startDate ? new Date(req.query.startDate) : void 0;
    const endDate = req.query.endDate ? new Date(req.query.endDate) : void 0;
    const sales2 = await storage.getSales(startDate, endDate);
    res.json(sales2);
  } catch (error) {
    res.status(500).json({ message: "Erro ao buscar vendas" });
  }
});
app2.get("/api/sales/:id", async (req, res) => {
  try {
    const sale = await storage.getSaleWithItems(req.params.id);
    if (!sale) {
      return res.status(404).json({ message: "Venda n\u00E3o encontrada" });
    }
    res.json(sale);
  } catch (error) {
    res.status(500).json({ message: "Erro ao buscar venda" });
  }
});

```

```

app2.post("/api/sales", async (req, res) => {
  try {
    const { items, ...saleData } = req.body;
    const validatedSale = insertSaleSchema.parse(saleData);
    const itemsSchema = z2.array(z2.object({
      inventoryId: z2.string(),
      quantity: z2.number().min(1),
      unitPrice: z2.number().min(0)
    })).min(1);
    const validatedItems = itemsSchema.parse(items);
    const sale = await storage.createSale(validatedSale, validatedItems);
    res.status(201).json(sale);
  } catch (error) {
    if (error instanceof z2.ZodError) {
      return res.status(400).json({ message: "Dados inv\xE1lidos", errors: error.errors });
    }
    res.status(500).json({ message: "Erro ao registrar venda" });
  }
});

app2.get("/api/financial-transactions", async (req, res) => {
  try {
    const startDate = req.query.startDate ? new Date(req.query.startDate) : void 0;
    const endDate = req.query.endDate ? new Date(req.query.endDate) : void 0;
    const type = req.query.type;
    const transactions = await storage.getFinancialTransactions(startDate, endDate, type);
    res.json(transactions);
  } catch (error) {
    res.status(500).json({ message: "Erro ao buscar transa\xE7\xF5es financeiras" });
  }
});

app2.post("/api/financial-transactions", async (req, res) => {
  try {
    const validatedData = insertFinancialTransactionSchema.parse(req.body);
    const transaction = await storage.createFinancialTransaction(validatedData);
    res.status(201).json(transaction);
  } catch (error) {
    if (error instanceof z2.ZodError) {
      return res.status(400).json({ message: "Dados inv\xE1lidos", errors: error.errors });
    }
    res.status(500).json({ message: "Erro ao registrar transa\xE7\xE3o financeira" });
  }
});

app2.get("/api/financial-summary", async (req, res) => {
  try {
    let startDate = /* @__PURE__ */ new Date();
    let endDate = /* @__PURE__ */ new Date();
    if (req.query.startDate && req.query.endDate) {

```

```

    startDate = new Date(req.query.startDate);
    endDate = new Date(req.query.endDate);
  } else {
    startDate = new Date(startDate.getFullYear(), startDate.getMonth(), 1);
    endDate = new Date(endDate.getFullYear(), endDate.getMonth() + 1, 0);
  }
  const summary = await storage.getFinancialSummary(startDate, endDate);
  res.json(summary);
} catch (error) {
  res.status(500).json({ message: "Erro ao buscar resumo financeiro" });
}
});
app2.get("/api/supabase/sync", async (req, res) => {
  try {
    const result = await syncDataWithSupabase();
    res.json(result);
  } catch (error) {
    res.status(500).json({ message: "Erro ao sincronizar com Supabase", error: error?.message });
  }
});
app2.get("/api/supabase/profiles", async (req, res) => {
  try {
    const { data, error } = await supabaseAdmin.from("profiles").select("*");
    if (error) {
      return res.status(500).json({ message: "Erro ao buscar perfis do Supabase", error: error.message });
    }
    res.json({
      profiles: data || [],
      count: data?.length || 0
    });
  } catch (error) {
    res.status(500).json({ message: "Erro ao acessar dados do Supabase", error: error?.message });
  }
});
app2.get("/api/supabase/users", async (req, res) => {
  try {
    const { data, error } = await supabaseAdmin.auth.admin.listUsers();
    if (error) {
      return res.status(500).json({ message: "Erro ao buscar usu\u00e1rios do Supabase", error: error.message });
    }
    res.json({
      users: data?.users || [],
      count: data?.users?.length || 0
    });
  } catch (error) {
    res.status(500).json({ message: "Erro ao acessar usu\u00e1rios do Supabase", error: error?.message });
  }
}

```

```

});
app2.get("/api/supabase/clients", async (req, res) => {
  try {
    const result = await getSupabaseClients();
    if (!result.success) {
      return res.status(500).json({
        message: "Erro ao buscar clientes do Supabase",
        error: result.error
      });
    }
    res.json({
      clients: result.clients,
      count: result.count
    });
  } catch (error) {
    console.error("Erro ao buscar clientes do Supabase:", error);
    res.status(500).json({
      message: "Erro ao acessar clientes do Supabase",
      error: error?.message || "Erro desconhecido"
    });
  }
});

app2.get("/api/supabase/appointments", async (req, res) => {
  try {
    const result = await getSupabaseAppointments();
    if (!result.success) {
      return res.status(500).json({
        message: "Erro ao buscar agendamentos do Supabase",
        error: result.error
      });
    }
    res.json({
      appointments: result.appointments,
      count: result.count
    });
  } catch (error) {
    console.error("Erro ao buscar agendamentos do Supabase:", error);
    res.status(500).json({
      message: "Erro ao acessar agendamentos do Supabase",
      error: error?.message || "Erro desconhecido"
    });
  }
});

app2.post("/api/supabase/set-service-key", async (req, res) => {
  try {
    const { serviceKey } = req.body;
    if (!serviceKey || typeof serviceKey !== "string" || serviceKey.length < 20) {

```

```

    return res.status(400).json({
      message: "Chave de servi\xE7o inv\xE1lida",
      error: "A chave de servi\xE7o deve ser uma string com pelo menos 20 caracteres"
    });
  }
  console.log("Chave de servi\xE7o recebida");
  return res.json({
    success: true,
    message: "Chave de servi\xE7o recebida com sucesso"
  });
} catch (error) {
  console.error("Erro ao configurar chave de servi\xE7o:", error);
  return res.status(500).json({
    message: "Erro ao configurar chave de servi\xE7o",
    error: error?.message || "Erro desconhecido"
  });
}
});
app2.post("/api/reset-demo-data", async (req, res) => {
  try {
    console.log("Solicita\xE7\xE3o recebida para limpar dados de demonstra\xE7\xE3o");
    return res.json({
      success: true,
      message: "Solicita\xE7\xE3o para limpar dados recebida",
      demo: true
      // Indica que esta é uma resposta de demonstração
    });
  } catch (error) {
    console.error("Erro ao limpar dados de demonstra\xE7\xE3o:", error);
    return res.status(500).json({
      message: "Erro ao limpar dados de demonstra\xE7\xE3o",
      error: error?.message || "Erro desconhecido"
    });
  }
});
const httpServer = createServer(app2);
return httpServer;
}

```

```

// server/vite.ts
import express from "express";
import fs from "fs";
import path2 from "path";
import { createServer as createViteServer, createLogger } from "vite";

```

```

// vite.config.ts
import { defineConfig } from "vite";

```

```

import react from "@vitejs/plugin-react";
import path from "path";
import runtimeErrorOverlay from "@replit/vite-plugin-runtime-error-modal";
var vite_config_default = defineConfig({
  plugins: [
    react(),
    runtimeErrorOverlay(),
    ...process.env.NODE_ENV !== "production" && process.env.REPL_ID !== void 0 ? [
      await import("@replit/vite-plugin-cartographer").then(
        (m) => m.cartographer()
      )
    ] : []
  ],
  resolve: {
    alias: {
      "@": path.resolve(import.meta.dirname, "client", "src"),
      "@shared": path.resolve(import.meta.dirname, "shared"),
      "@assets": path.resolve(import.meta.dirname, "attached_assets")
    }
  },
  root: path.resolve(import.meta.dirname, "client"),
  build: {
    outDir: path.resolve(import.meta.dirname, "dist/public"),
    emptyOutDir: true
  }
});

```

```

// server/vite.ts
import { nanoid } from "nanoid";
var viteLogger = createLogger();
function log(message, source = "express") {
  const formattedTime = (/* __PURE__ */ new Date()).toLocaleTimeString("en-US", {
    hour: "numeric",
    minute: "2-digit",
    second: "2-digit",
    hour12: true
  });
  console.log(`${formattedTime} [${source}] ${message}`);
}
async function setupVite(app2, server) {
  const serverOptions = {
    middlewareMode: true,
    hmr: { server },
    allowedHosts: true
  };
  const vite = await createViteServer({
    ...vite_config_default,

```

```

configFile: false,
customLogger: {
  ...viteLogger,
  error: (msg, options) => {
    viteLogger.error(msg, options);
    process.exit(1);
  }
},
server: serverOptions,
appType: "custom"
});
app2.use(vite.middlewares);
app2.use("*", async (req, res, next) => {
  const url = req.originalUrl;
  try {
    const clientTemplate = path2.resolve(
      import.meta.dirname,
      "..",
      "client",
      "index.html"
    );
    let template = await fs.promises.readFile(clientTemplate, "utf-8");
    template = template.replace(
      `src="/src/main.tsx"`,
      `src="/src/main.tsx?v=${nanoid()}"`
    );
    const page = await vite.transformIndexHtml(url, template);
    res.status(200).set({ "Content-Type": "text/html" }).end(page);
  } catch (e) {
    vite.ssrFixStacktrace(e);
    next(e);
  }
});
}
function serveStatic(app2) {
  const distPath = path2.resolve(import.meta.dirname, "public");
  if (!fs.existsSync(distPath)) {
    throw new Error(
      `Could not find the build directory: ${distPath}, make sure to build the client first`
    );
  }
  app2.use(express.static(distPath));
  app2.use("*", (_req, res) => {
    res.sendFile(path2.resolve(distPath, "index.html"));
  });
}
}

```



```
// server/initData.ts
async function initializeDatabase() {
  console.log("Verificando a conex\u00e3o com o banco de dados do Supabase...");
  try {
    const { count, error } = await supabase.from("clients").select("*", { count: "exact", head: true });
    if (error) {
      throw new Error(`Erro ao conectar ao Supabase: ${error.message}`);
    }
    console.log(`Conex\u00e3o com o Supabase estabelecida com sucesso. Existem ${count} clientes cadastrados.`);
    const tabelas = ["clients", "appointments", "services", "inventory"];
    for (const tabela of tabelas) {
      const { count: count2, error: error2 } = await supabase.from(tabela).select("*", { count: "exact", head: true });
      if (error2) {
        console.warn(`Aviso: N\u00e3o foi poss\u00edvel acessar a tabela ${tabela}: ${error2.message}`);
      } else {
        console.log(`Tabela ${tabela}: ${count2} registros encontrados`);
      }
    }
    console.log("Inicializa\u00e7\u00e3o do banco de dados conclu\u00edda.");
  } catch (error) {
    console.error("Erro durante a inicializa\u00e7\u00e3o do banco de dados:", error);
    console.error("ATEN\u00c7\u00c3O: A conex\u00e3o com o banco de dados do Supabase falhou.");
    console.error("Verifique as credenciais (SUPABASE_URL e SUPABASE_KEY) e a conectividade de rede.");
  }
}
```

```
// server/index.ts
var app = express2();
app.use(express2.json());
app.use(express2.urlencoded({ extended: false }));
app.use((req, res, next) => {
  const start = Date.now();
  const path3 = req.path;
  let capturedJsonResponse = void 0;
  const originalResJson = res.json;
  res.json = function(bodyJson, ...args) {
    capturedJsonResponse = bodyJson;
    return originalResJson.apply(res, [bodyJson, ...args]);
  };
  res.on("finish", () => {
    const duration = Date.now() - start;
    if (path3.startsWith("/api")) {
      let logLine = `${req.method} ${path3} ${res.statusCode} in ${duration}ms`;
      if (capturedJsonResponse) {
        logLine += ` :: ${JSON.stringify(capturedJsonResponse)}`;
      }
      if (logLine.length > 80) {
```

```

        logLine = logLine.slice(0, 79) + "\u2026";
    }
    log(logLine);
}
});
next();
});
(async () => {
    await initializeDatabase();
    const server = await registerRoutes(app);
    app.use((err, _req, res, _next) => {
        const status = err.status || err.statusCode || 500;
        const message = err.message || "Internal Server Error";
        res.status(status).json({ message });
        throw err;
    });
    if (app.get("env") === "development") {
        await setupVite(app, server);
    } else {
        serveStatic(app);
    }
    const port = 5e3;
    server.listen({
        port,
        host: "0.0.0.0",
        reusePort: true
    }, () => {
        log(`serving on port ${port}`);
    });
})();

```